



How valyou Works

A tour of every moving part in a program that reads a social media feed faster than you can blink and hides the cruelty before it reaches your eyes — without ever sending a single word to anybody.

ENGINEERING ARCHITECTURE NO PRIOR ENGINEERING KNOWLEDGE ASSUMED

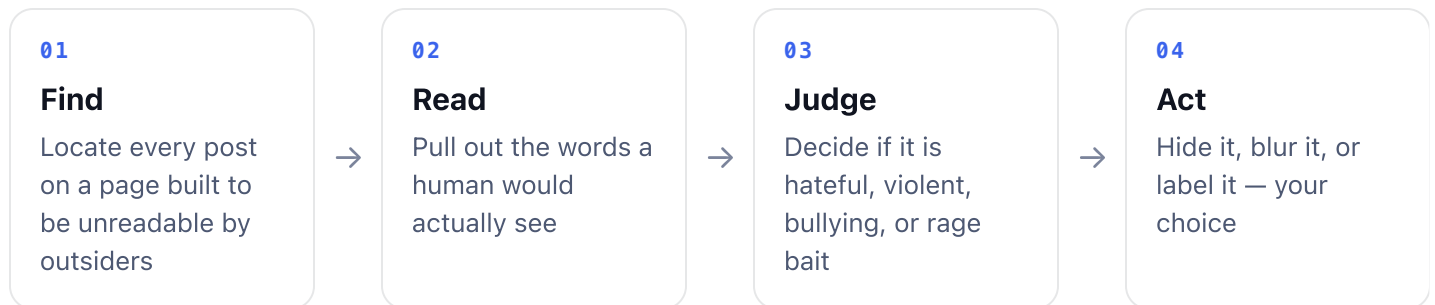
SUNAYU LLC · 2026 GPL-3.0-OR-LATER

START HERE

The problem, in one paragraph

Social media apps decide what you see. They are very good at showing you things that make you angry, because angry people keep scrolling, and scrolling makes money. valyou is a small program that sits between the feed and your eyes and quietly removes the worst of it — racist and sexist hate, threats of violence, bullying, and posts engineered to enrage you.

That sentence hides an enormous amount of engineering. To pull it off, valyou has to do six hard things in the time it takes a post to slide onto your screen:



The whole pipeline, minus the details. Steps 1–4 run in about a quarter of a millisecond per post — roughly a thousand times faster than an eye blink.

And two more that are easy to forget and impossible to skip: it must **never phone home** (nothing you read may leave your device), and it must **be wrong gracefully** — because a filter that hides your grandmother's birthday post gets uninstalled by dinnertime.

THE BIG ANALOGY

Imagine a librarian who stands beside you while you flip through a magazine. She reads every page a half-second before you do. If a page is full of cruelty, she slips a sheet of frosted glass over it and writes a small note explaining why. She never tells anyone what you read. She never leaves the room. That librarian is valyou, and this document explains exactly how she is built.

CHAPTER 1

Finding the posts

Before valyou can judge a post, it has to find one. This turns out to be the single hardest part of the whole system, and it is where almost every bug has lived.

What a web page actually is

A web page is not a picture. It is a **tree** of boxes inside boxes, called the **DOM** (Document Object Model). A post is one branch of that tree — a box containing the author's name, the text, maybe a photo, and the buttons.

If Facebook labeled that box `<div class="post">`, this chapter would be one sentence long. They don't. Facebook's boxes have machine-generated names like `class="x1lllihq x1n2onr6"` that **change every few weeks**. Anything built on those names breaks constantly.

THE TRICK

There is one part of the page they *can't* scramble: the labels that screen readers use so blind people can browse. Those are called **ARIA** attributes — things like `role="article"` for a post and `role="feed"` for the scrolling list. Breaking them would make the site unusable for blind users, which is both wrong and illegal in many places. So valyou anchors everything on ARIA. It is the one honest handhold on a slippery wall.

Six rules, tried in order

valyou keeps a short list of patterns, from most specific to most desperate, and tries each in turn. Later rules politely step aside when an earlier one already claimed a chunk of the page.

#	WHAT IT LOOKS FOR	WHY IT EXISTS
1	A chat row	Messenger conversations
2	<code>role="article"</code>	The normal feed post — the reliable case
3	A numbered item in the feed	Suggested posts and ads that skip the article label
4	A plain <code><article></code> tag	The phone version of the site
5	A pop-up dialog	The full-screen "theater" view when you tap a video
6	Any block of user-written text	The last resort — see below

Why rule 6 exists (a story about losing)

Rules 2 through 5 are each a *guess* about which wrapper Facebook used today. A user reported that a post with a slur in it wasn't being hidden. It was fixed. It was still broken. Fixed again. Still broken. Three times.

Each fix targeted the shell Facebook happened to use in that one view — the feed, then the pop-up, then the reshare. Facebook has many shells and changes them whenever it likes. Chasing them is a losing game.

THE LESSON

Stop chasing the *container* and match the *content*. Facebook marks every piece of human-written text with the same little flag — `dir="auto"`, which tells the browser "figure out if this language reads left-to-right or right-to-left." Every caption has it, in every shell, because it's about language, not layout. Rule 6 matches that. A caption is now caught no matter which box Facebook wraps it in.

Rule 6 is bounded by two guards so it can't run wild: it ignores text that belongs to a post another rule already judged, and it never touches the **site's own furniture** — the search box, menus, navigation. That second guard exists because of a genuinely funny bug: searching for a slur (to test the filter!) caused valyou to blur *your own search box*, because your typed word was, technically, hateful text on the page.

CHAPTER 2

Reading the words

Now that we have the box, we need the words inside it — exactly the words a human would see, no more and no less.

The obvious approach is to ask the browser "give me all the text in this box." That fails for a subtle reason: it would also scoop up the reply you're half-typing in the comment field, and valyou's own labels, and count them as part of the post. So instead valyou walks the tree itself, skipping the places where *you* type and the labels it added.

The bug that hid in plain sight for weeks

The original walker read text only at the **leaves** of the tree — the deepest boxes with nothing inside them. That's a normal, sensible design. It was also silently wrong.

```
<!-- What Facebook actually sends: -->
<div>
  "You're not a [slur], you're a person."  ← bare text, no box!
  <span>😂😂</span>                        ← a leaf
  <span>😂😂</span>                        ← a leaf
</div>
```

The caption is a bare piece of text sitting *beside* the emoji boxes, not inside one. A leaf-only reader collects "😂😂" and misses the entire sentence.

So valyou was reading a post's timestamp, the author's name, and two laughing emoji — and concluding, quite reasonably, that nothing was wrong. The fix is one idea: **read text wherever it lives**, whether or not it has a box of its own. Programmers call this "mixed content," and it is everywhere once you look.

DETAIL THAT MATTERS

valyou also reads the **alt text** of images — the description platforms generate for blind users ("May be an image of two people and text that says..."). It's free extra evidence, already sitting in the page, and it is often the only clue about a photo.

CHAPTER 3

Judging

valyou now has a paragraph of text and must produce five numbers: how racist, how sexist, how violent, how harassing, how rage-baiting is this?

Two judges, not one

There are two completely different ways to judge text, and valyou uses both, for reasons that come down to trust.

JUDGE ONE **The rulebook**

A curated list of slurs, threats, dehumanizing phrases ("they're animals"), and rage-bait shapes (ALL CAPS, "you won't BELIEVE"). Exact, explainable, and it never surprises you. But it only knows what it was told.

JUDGE TWO **The learning machine**

A model trained on half a million real examples. It catches hostile phrasing nobody wrote a rule for. But it cannot explain itself, and it inherited every bias in its training data.

Adding up evidence without double-counting

Suppose a post contains a slur (strong evidence, 0.9) and is also written entirely in capitals (weak evidence, 0.3). You can't add those — you'd get 1.2, and probabilities can't exceed 1. You can't take the maximum either, because two pieces of evidence really are worse than one.

valyou uses a rule called **noisy-OR**, which asks the question backwards:

```
// What's the chance EVERY signal is wrong?  
both wrong = (1 - 0.9) × (1 - 0.3) = 0.10 × 0.70 = 0.07  
  
// So the chance at least one is right:  
score = 1 - 0.07 = 0.93
```

Each new piece of evidence pushes the score closer to 1 but never past it. Weak signals stack up; strong ones dominate. This is the same math used to combine independent tests in medicine.

Your dials, not ours

A score only becomes an action after it clears a **threshold** you control. Each category has its own dial and its own choice of what to do — hide it completely, blur it behind frosted glass, or just tag it and leave it visible. Slide the dial down and valyou gets stricter; slide it up and it relaxes.

CHAPTER 4

Inside the learning machine

This is the part people call "the AI." It is smaller and much less mysterious than that word suggests: it is a long list of numbers and one multiplication.

Step 1 — chop the text into clues

A computer can't read. So valyou chops the text into overlapping fragments called **n-grams** and treats each as a yes/no clue:

KIND	EXAMPLE FROM "YOU PEOPLE"	WHAT IT CATCHES
Single words	you , people	Topic
Word pairs	you people	Phrasing — the pair is hostile, neither word is
Letter runs (3–5)	peo , eopl , peopl	Misspellings and disguises: h8 , n1gger , h.a.t.e

Those letter-runs are why deliberate misspelling doesn't work as an escape hatch. Change a letter and most fragments survive.

Step 2 — the hashing trick

There are millions of possible fragments. Storing a dictionary of them would be huge. So valyou runs each fragment through a **hash function** — a blender that turns any text into a number — and keeps just 65,536 slots.

ANALOGY

Like a coat check with 65,536 hooks and a rule for which hook each coat goes on. Occasionally two coats share a hook (a "collision"). For this kind of model that's a tiny amount of noise, and it's a fantastic trade for never needing the dictionary at all.

Step 3 — add up the weights

Each of the 65,536 slots holds a **weight**: a number learned from training. Positive means "this fragment suggests hate," negative means the opposite. To score a post, valyou adds the weights of the fragments present, then squashes the total into a 0–1 probability with an S-shaped curve called a **sigmoid**.

```
score = sigmoid( bias +  $\sum$  weight[fragment] )
```

```
where sigmoid(z) = 1 / (1 + e-z)
```

```
big positive z → near 1      big negative z → near 0
```

That's the entire model. One addition per fragment, one curve at the end. It runs in about 18 microseconds — 0.000018 seconds.

Step 4 — five judges, one reading

Version 1 had a single "is this hateful?" score, which had to be smeared across three categories. Version 2 has **five separate sets of weights** — one per harm — that all share the same fragment-chopping work. Racist language now strengthens the racism score specifically.

JUDGE	TRAINED ON	SKILL (AUC)	ACCURACY WHEN IT SPEAKS
Racist language	Jigsaw civil comments, HateXplain, Davidson	0.967	90%
Sexist language	EDOS (SemEval 2023), HateXplain	0.897	90%
Violence & threats	Jigsaw threats, EDOS threats	0.970	97%
Harassment	Jigsaw insults, Davidson	0.981	98%
Rage bait	Stop Clickbait corpus	0.995	98%

AUC means: pick one hateful post and one clean post at random; how often does the model rank the hateful one higher? 0.5 is a coin flip. 1.0 is perfect. Trained on roughly **530,000** labeled examples, up from 25,000 in version 1.

Step 5 — shrink it to fit

65,536 weights × 5 judges as full-precision numbers would be over a megabyte. Instead each weight is squeezed into a **single byte** (a number from −127 to 127) plus one shared scaling factor. That's **quantization**: about four times smaller, for accuracy loss too small to notice.

THE MOST IMPORTANT RULE IN THIS DOCUMENT

The learning machine is **never allowed to decide alone**. Its influence is capped at 0.5, and every default action threshold is higher (0.55, 0.6, 0.7). So on default settings, the model can strengthen evidence the rulebook already found — it can never hide a post by itself.

This is deliberate. Hate-speech training data carries known biases; models trained on it disproportionately flag African American English and posts by the very people they're meant to protect. valyou's answer isn't to claim the bias away — it's to *bound* it. A biased hint is survivable. A biased judge is not. Automated tests enforce that cap on every single build.

CHAPTER 5

Doing something about it

A verdict is worthless until the page changes. This is the only part of valyou allowed to modify what you see — deliberately kept separate from everything that reads.

Hide

Collapses to a slim bar naming the reason.
Reversible in one tap.

Blur

Frosted glass over the post.
You see something is there and choose.

Tag

A small label, nothing covered. For "I want to know, not be protected."

Covering the right box

Early versions covered the post and left the **headline above it** perfectly readable — because Facebook puts the header *beside* the post box, not inside it. Covering the body but not the title is worse than useless.

So the cover grows outward to the largest box that still contains **only this post**, with three hard stops so it can never swallow a neighbour: stop at the feed container, stop if another post appears inside, stop at the site's own furniture.

THE TRAP INSIDE THE FIX

The first version of that growth rule stopped whenever it saw more than one post-shaped box in the parent. It never grew at all — because Facebook nests *comment previews* (which are also post-shaped) inside every post. The rule now ignores boxes nested inside our own post and only stops for genuine outsiders.

CHAPTER 6

Stopping video before it starts

Video is a different beast: by the time you've read a caption, an autoplaying clip has already shown you fifty frames.

The obvious approach — wait for the video to start, then pause it — loses. You always see a flicker, and modern players hide inside **shadow DOM**, a sealed compartment whose events never reach the outside. *valyou* would pause it, the player would retry, and the result looked like continuous playback.

Locking the doors instead

valyou's engine runs inside the page's own JavaScript world, which means it can change the browser's own machinery before the site uses it. There are exactly four ways a video can start, and all four are now guarded:

DOOR	HOW THE SITE OPENS IT	THE GUARD
1	Calls <code>play()</code> in code	The function itself is replaced; it politely refuses
2	Sets the autoplay switch	The switch is intercepted and ignored
3	Writes the autoplay label onto the tag	Same interception, different spelling
4	Ships HTML that already says "autoplay"	A watcher strips it the instant it appears

Facebook's feed uses door 4. Facebook's Reels use door 1. Guarding only one leaves the other wide open — which is exactly what happened, twice.

THE POLITE REFUSAL

When valyou blocks a play, it hands back the exact same error the browser gives in Low Power Mode. Facebook's player already knows that error: it shows a play button and stops trying. No crash, no infinite retry loop — the site thinks the phone is saving battery.

Making your tap work anyway

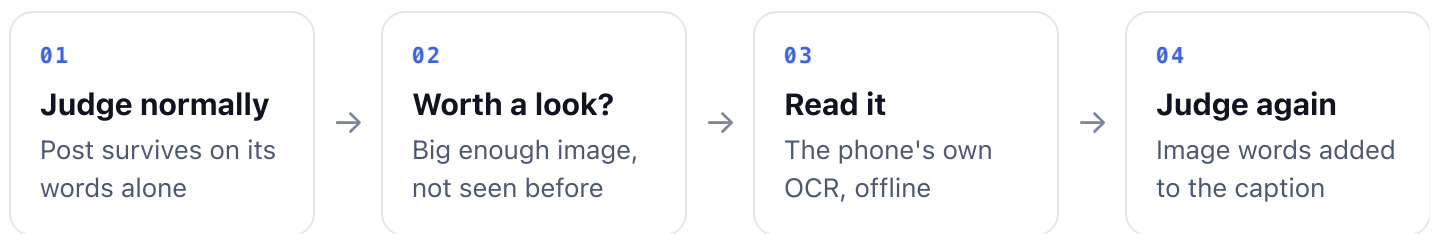
Blocking autoplay is only half the job; the video must play when *you* ask. That's harder than it sounds, because websites react to your finger lifting (`pointerup`) a few milliseconds *before* the browser reports a click. By the time valyou knows you tapped, the site already tried to play, already got refused, and has given up.

So valyou starts the video **itself**: it works out which player you touched, approves that one video, un-mutes it, and plays it — then nudges once more 350ms later in case the site reset things. It also opens a 15-second grace window that survives page changes, so the video keeps playing when Facebook re-creates it in full-screen view.

Reading the words inside pictures

A huge share of online hate is a meme: the cruelty is painted into the image, where no text reader can see it.

On phones, valyou hands the image to the operating system's built-in text recognizer (Apple's Vision framework; Android's ML Kit next). Both run **entirely offline** — no model to download, nothing sent anywhere — so this costs no app size and breaks no promises.



OCR runs only on posts that were *not* already filtered, so it can add protection but never remove it.

The image's words are **appended to the post's own text** rather than judged alone. That matters: a word on a T-shirt in a family photo gets read in context, instead of standing alone looking like a statement.

Every part of this is bounded so a bottomless feed can't turn it into a battery problem: results cached per image, 40 images per page, nothing under 160 pixels, one at a time, hard time limits, and low-confidence readings thrown away — because a misread word could invent a slur that was never there.

CHAPTER 8

Why nothing leaks

Most apps promise privacy in a policy document. valyou is built so that breaking the promise would require rewriting the program.

No network permission

The browser extension never requests the ability to send data. The browser itself enforces it — not a policy, a wall.

Text is never stored

A verdict is computed and the words are thrown away in the same instant.

Counters, not history

"41 filtered today" — a number, with nothing attached about what or whose.

Encrypted at rest

Even your settings are locked with AES-256-GCM, keyed by hardware (Secure Enclave / Android Keystore), device-only, never cloud-synced.

WHY THIS DESIGN, NOT A SERVER

A server would be easier to build and would make a better demo. It would also mean a database somewhere containing every hateful thing every user ever scrolled past — a target worth stealing and a subpoena worth serving. The safest data is data that was never collected.

There is one more layer: the web page is treated as an **enemy**. Settings live in native storage and are pushed *into* the page read-only; only counters come back. Even if a site attacked valyou's own code, there is nothing there worth taking.

CHAPTER 9

One brain, four bodies

valyou runs in a desktop browser, in Safari, on iPhone, and on Android. Writing that logic four times would guarantee four different

sets of bugs.

Shells — Chrome/Brave extension · Safari extension · iPhone app · Android app

The engine (shared, identical everywhere) — find posts · read text · rulebook · five-judge model
· hide/blur/tag · video guards

Native services — encrypted settings · image text recognition

Every shell loads the same engine. A fix made once is a fix everywhere; a bug found on the phone is a bug fixed in the browser too.

On phones the shell is a **browser**, because neither iOS nor Android lets one app reach inside another. A filtering browser is the only design that can work and still be allowed in the stores.

A BUG WORTH REMEMBERING

On Android, valyou filtered **nothing** — silently. The engine is handed to the page as one long string of text, and Android's app-to-page channel has a size limit of about one megabyte. Our engine, in the format Android counts, weighed 1.3 MB. The system dropped it with no error, no warning, nothing.

The fix: send it in slices and reassemble them inside the page. The lesson: **silence is not success**. Now the tests prove the engine is present, rather than assuming.

CHAPTER 10

How we know it works

Every claim in this document is checked by a machine, repeatedly, because humans are bad at noticing what silently stopped working.

Roughly **260 automated tests** run on every change. They check the arithmetic, the text handling, the encryption — and, most importantly, the promises:

- The learning machine's influence stays below every action threshold
- Clean everyday posts don't get filtered
- Each historical bug in this document stays fixed — one test per bug

Testing against the real thing

Tests with fake pages can't catch Facebook changing its HTML. So valyou also has a harness that launches a real browser, loads the real extension, opens a real public post, and reports what happened.

THE RULE THAT CAME FROM BEING WRONG

Count what a *human* would see, not what the code believes. Early checks counted "posts marked hidden" and looked healthy while a second copy of the same post sat visible on screen. Now the harness measures actual pixel size and visibility. If it is 0 pixels tall, it's gone. If it isn't, we aren't done.

APPENDIX A

The numbers

THING	VALUE	WHY IT MATTERS
Time to judge one post	~0.25 ms	Faster than one screen refresh — you never see a delay
Time for all five model judges	~0.018 ms	They share the chopping work, so five cost barely more than one
Model slots	65,536	Fragment storage, per judge
Model size on disk	~430 KB	Small enough to ship inside the app
Training examples	~530,000	Up from 25,000 in version 1
Automated tests	~260	Run on every change
Network requests made	0	The whole point

APPENDIX B

Glossary

DOM (Document Object Model)

The browser's tree of boxes that makes up a page. Programs read and change the page through it.

ARIA

Labels that describe what parts of a page are *for* so screen readers can work. Stable, because breaking them breaks accessibility.

Shadow DOM

A sealed compartment inside a page. Ordinary searching can't see in — which is where video players like to hide.

n-gram

A run of n things in a row — three letters, two words. valyou's clue format.

Hash function

A blender that turns any text into a number, always the same number for the same text.

Logistic regression

The model type here: multiply clues by learned weights, add them, squash to a probability.

Sigmoid

The S-curve that turns any number into a 0-to-1 probability.

AUC

A score for ranking ability: given one bad and one good example, how often is the bad one ranked higher? 0.5 = guessing, 1.0 = perfect.

Quantization

Storing numbers with less precision to save space — here, one byte per weight.

Noisy-OR

Combining independent evidence by computing the chance that *all* of it is wrong.

OCR

Optical Character Recognition — reading letters out of a picture.

AES-256-GCM

A strong encryption method that also detects tampering.

valyou — on-device social media content filtering · © 2026 Sunayu LLC · Free software under the GNU GPL v3.0 or later, with an App Store distribution exception. Source: valyou.cx

Written for anyone curious enough to ask how it actually works — every mechanism explained from first principles, no jargon left undefined.